

RECOMENDACIÓN DE FICHAJES IDEALES

**Presentación Trabajo
Final**

Laura Carrasco Sánchez
Pablo Montero Tejero

CREACIÓN DE UNA APLICACIÓN PARA LA RECOMENDACIÓN DE FICHAJES IDEALES PARA REEMPLAZAR JUGADORES EN LA NBA

INTELIGENCIA ARTIFICIAL Y ESTADÍSTICA

https://github.com/PabloM004/Trabajo_IAE.git

CERTIFICADOS

Laura Carrasco Sánchez

STATEMENT OF ACCOMPLISHMENT

#46,398,691

HAS BEEN AWARDED TO

Pablo Montero

FOR SUCCESSFULLY COMPLETING

Python for R Users

LENGTH

5 HRS

COMPLETED ON

FEB 25, 2026

 datacamp

STATEMENT OF ACCOMPLISHMENT

#46,534,803

HAS BEEN AWARDED TO

Pablo Montero

FOR SUCCESSFULLY COMPLETING

Joining Data with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 02, 2026

 datacamp



Jonathan Cornelissen
CEO, DataCamp

STATEMENT OF ACCOMPLISHMENT

#46,542,296

HAS BEEN AWARDED TO

Pablo Montero

FOR SUCCESSFULLY COMPLETING

Data Manipulation with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 04, 2026

 datacamp



Jonathan Cornelissen
CEO, DataCamp

CERTIFICADOS



Certificate of Completion

Dagster Essentials

This certificate is awarded to
Pablo Montero Tejero

Issued: 2026-04-26



STATEMENT OF ACCOMPLISHMENT

#46,671,949

HAS BEEN AWARDED TO

Pablo Montero

FOR SUCCESSFULLY COMPLETING

Writing Efficient Code with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 16, 2026



STATEMENT OF ACCOMPLISHMENT

#47,066,732

HAS BEEN AWARDED TO

Pablo Montero

FOR SUCCESSFULLY COMPLETING

Reshaping Data with pandas

LENGTH

4 HRS

COMPLETED ON

APR 03, 2026

 datacamp



Jonathan Cornelissen

Pablo Montero Tejero



STATEMENT OF ACCOMPLISHMENT

#46,398,691

HAS BEEN AWARDED TO

Pablo Montero

FOR SUCCESSFULLY COMPLETING

Python for R Users

LENGTH

5 HRS

COMPLETED ON

FEB 25, 2026



STATEMENT OF ACCOMPLISHMENT

#46,542,296

HAS BEEN AWARDED TO

Pablo Montero

FOR SUCCESSFULLY COMPLETING

Data Manipulation with pandas

LENGTH

4 HRS

COMPLETED ON

MAR 04, 2026



Jonathan Cornelissen
CEO, DataCamp



STATEMENT OF ACCOMPLISHMENT

#46,534,803

HAS BEEN AWARDED TO

Pablo Montero

FOR SUCCESSFULLY COMPLETING

Joining Data with pandas

LENGTH

4 HRS

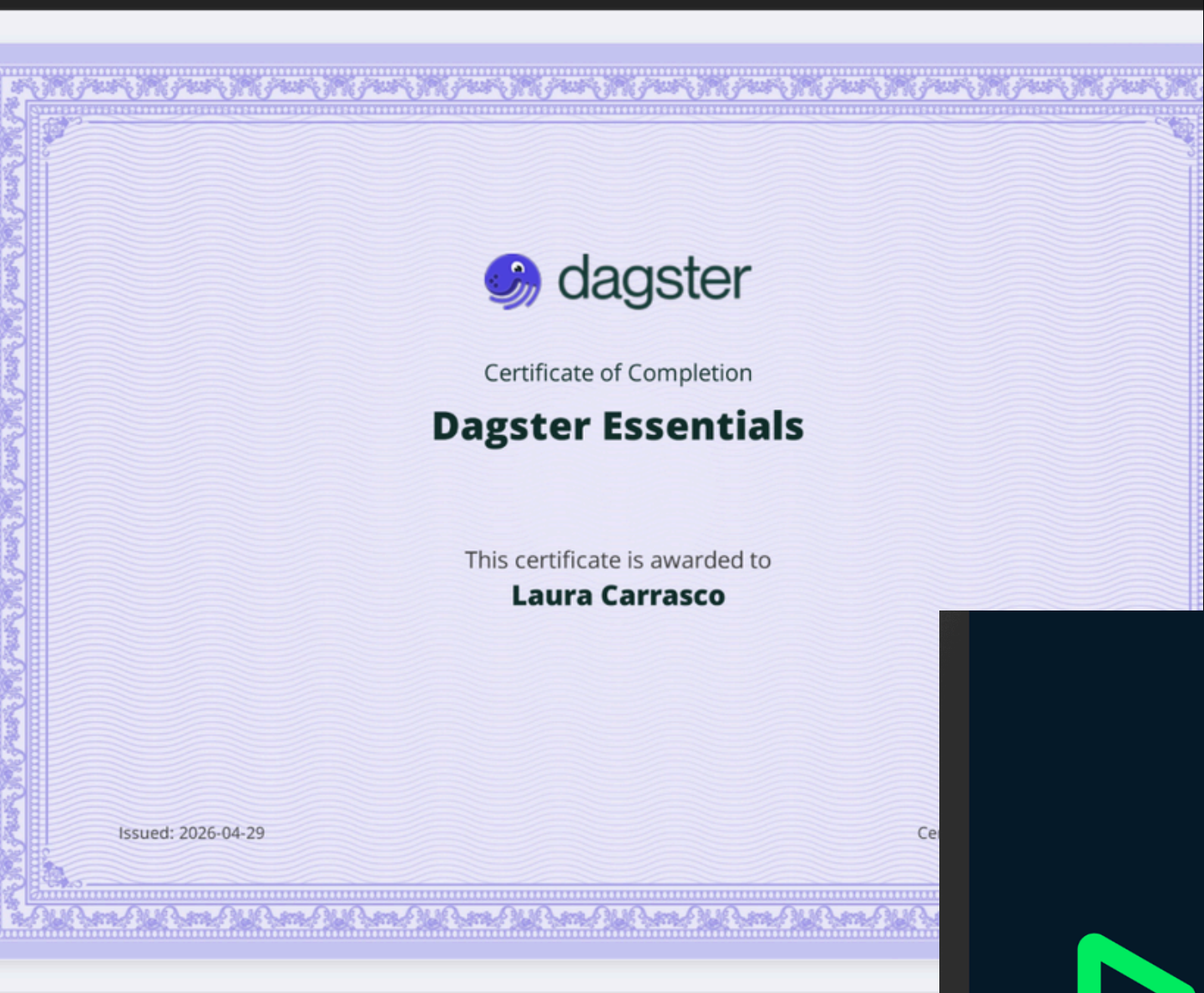
COMPLETED ON

MAR 02, 2026



Jonathan Cornelissen
CEO, DataCamp

CERTIFICADOS



STATEMENT OF ACCOMPLISHMENT

#47,448,001

HAS BEEN AWARDED TO

laucarsan

FOR SUCCESSFULLY COMPLETING

Writing Efficient Code with pandas

LENGTH

4 HRS

COMPLETED ON

APR 27, 2026

A handwritten signature in black ink, located at the bottom right of the certificate.

STATEMENT OF ACCOMPLISHMENT

#47,479,146

HAS BEEN AWARDED TO

laucarsan

FOR SUCCESSFULLY COMPLETING

Reshaping Data with pandas

LENGTH

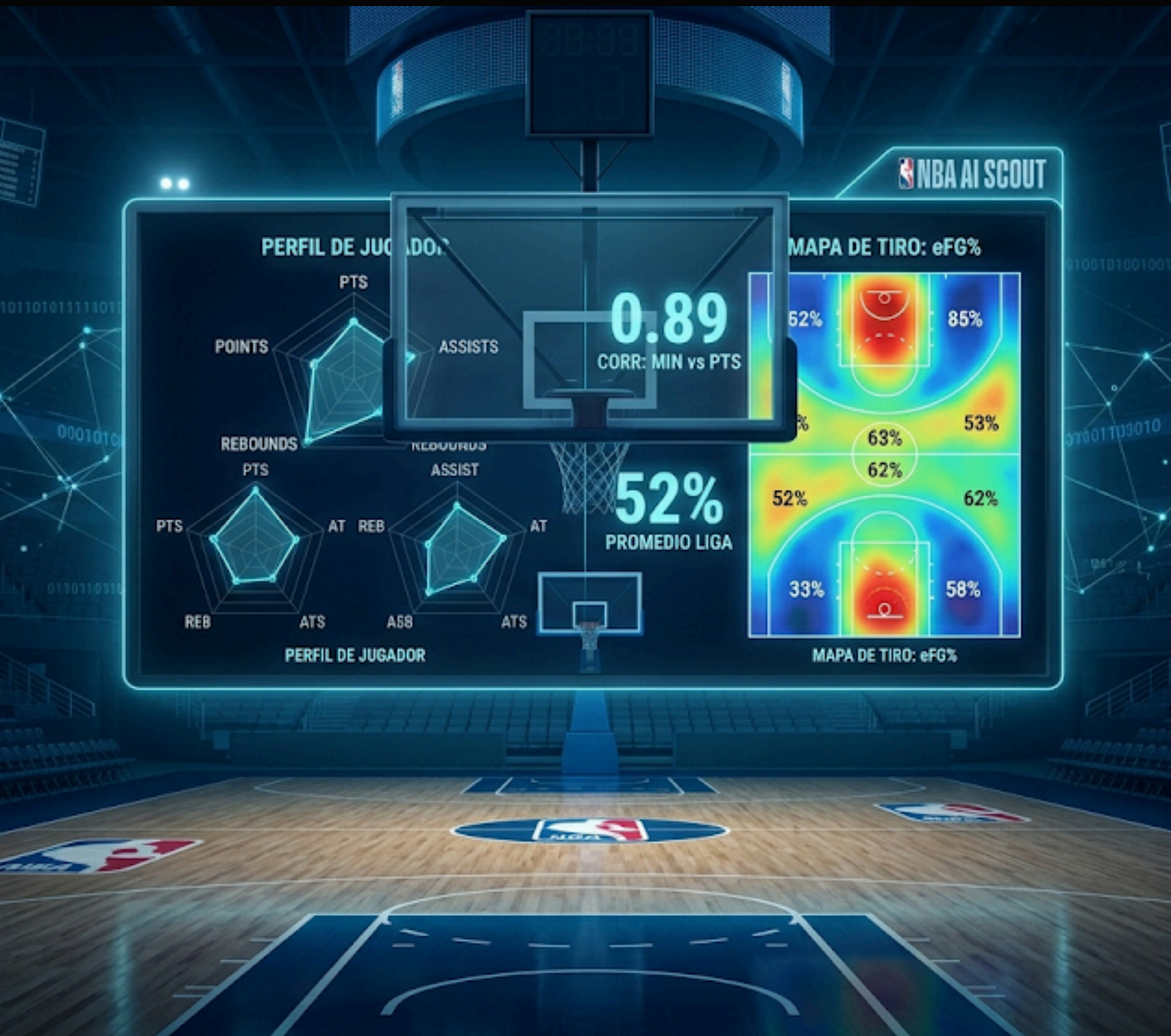
4 HRS

COMPLETED ON

APR 29, 2026

A handwritten signature in black ink, located at the bottom right of the certificate.





CONTEXTO

Durante cada temporada de la NBA cada equipo disputa 82 partidos fijos, con posibilidad de sumar hasta 20 partidos extras si compete en la post-temporada. Dicha carga de partidos puede suponer:

- **Lesiones desafortunadas de jugadores, tanto estrellas como jugadores de rol.**
- **Fallos de confección de plantilla, las cuales pueden hacer que el equipo no rinda como se esperaba de él.**

Por ello, hemos pensado en realizar una aplicación en la cual, seleccionando un jugador cualquiera (sea lesionado o por decisión de la directiva del equipo), se pueda encontrar el reemplazo que mejor se ajuste a su rol en el equipo.

DATOS

<https://www.kaggle.com/datasets/javigallego/stats-nba>



FRANCISCO JAVIER GALLEGO · UPDATED
4 YEARS AGO

▲ 26

<> Code

Download

⋮

NBA Stats since 1980

NBA players and teams stats since 1980 to 2022 season. Includes mvp voting data.



1980 – 2021

- **INDICE:** IDENTIFICADOR NUMÉRICO ÚNICO DE LA FILA.
- **JUGADOR:** NOMBRE COMPLETO DEL DEPORTISTA.
- **ALTURA:** ESTATURA DEL JUGADOR
- **PESO:** PESO DEL JUGADOR
- **UNIVERSIDAD:** CENTRO EDUCATIVO DE PROCEDENCIA ANTES DE ENTRAR EN LA LIGA.
- **POSICION:** ROL TÁCTICO EN LA CANCHA
- **EDAD:** EDAD DEL JUGADOR EN LA TEMPORADA CORRESPONDIENTE.
- **TEMPORADA:** AÑO CORRESPONDIENTE A LOS REGISTROS ESTADÍSTICOS.
- **PARTIDOS JUGADOS:** NÚMERO TOTAL DE ENCUENTROS EN LOS QUE PARTICIPO.
- **PARTIDOS TITULAR:** NÚMERO DE ENCUENTROS EN LOS QUE FORMO PARTE DEL QUINTETO INICIAL.
- **MINUTOS JUGADOS_PP:** PROMEDIO DE MINUTOS JUGADOS POR PARTIDO.
- **TIROS ANOTADOS_PP:** PROMEDIO DE TIROS DE CAMPO ENCESTADOS POR PARTIDO.
- **PORCENTAJE TIROS EFECTIVO:** MÉTRICA AJUSTADA QUE OTORGA UN VALOR EXTRA A LOS TRIPLES (EFG%).
- **TIROS LIBRES ANOTADOS:** PROMEDIO DE TIROS LIBRES ENCESTADOS POR PARTIDO.
- **PUNTOS_PP:** PROMEDIO DE PUNTOS TOTALES ANOTADOS POR PARTIDO.
- **ASISTENCIAS_PP:** PASES REALIZADOS QUE TERMINAN DIRECTAMENTE EN CANASTA.
- **SRS: SIMPLE RATING SYSTEM:** MÉTRICA QUE CALIFICA AL EQUIPO CONSIDERANDO EL MARGEN DE VICTORIA Y LA DIFICULTAD DEL CALENDARIO.
- ...

TEMAS APLICADOS

1. DATACAMP: PYTHON FOR R USERS

Traslado de lenguajes de programación

2. DATACAMP: JOIN DATA WITH PANDAS

Saber cómo importar, leer e interpretar las salidas de los datos

```
df = pd.read_csv('cleaned_data.csv')  
print(df.head())
```

	Unnamed: 0	Player	Ht	Wt	Colleges	Pos	Age	Tm	G	GS	\
0	0	Alaa Abdelnaby	6-10	240.0	Duke	PF	22	POR	43	0	
1	1	Danny Ainge	6-4	175.0	BYU	SG	31	POR	80	0	
2	2	Mark Bryant	6-9	245.0	Seton Hall	PF	25	POR	53	0	
3	3	Wayne Cooper	6-10	220.0	New Orleans	C	34	POR	67	1	
4	4	Walter Davis	6-6	193.0	UNC	SG	36	POR	71	14	

	...	Pts	Max	Share	Team	W	L	W/L%	GB	PS/G	\
0	...		0.0	0.0	Portland Trail Blazers	63	19	0.768	0.0	114.7	
1	...		0.0	0.0	Portland Trail Blazers	63	19	0.768	0.0	114.7	
2	...		0.0	0.0	Portland Trail Blazers	63	19	0.768	0.0	114.7	
3	...		0.0	0.0	Portland Trail Blazers	63	19	0.768	0.0	114.7	
4	...		0.0	0.0	Portland Trail Blazers	63	19	0.768	0.0	114.7	

	PA/G	SRS
0	106.0	8.47
1	106.0	8.47
2	106.0	8.47
3	106.0	8.47
4	106.0	8.47

3. DATACAMP: DATA MANITULATION WITH PANDAS

Realizar transformaciones de nuestra base de datos para que esta sea lo más óptima posible

```
df = df.drop('Indice', axis=1)
```

```
cols_eliminar= [
    'Tiros_Anotados_PP', 'Tiros_Intentados_PP', 'Triples_Anota
    'Dobles_Intentados_PP', 'Tiros_Libres_Anotados', 'Tiros_L
    'Rebotes_Ofensivos_PP', 'Rebotes_Defensivos_PP',
    'Puntos_MVP', 'Puntos_MVP_Obtenebles']
df=df.drop(columns=cols_eliminar)
```

```
df = df.drop('Indice', axis=1)
```

- Eliminar columnas
- Existencia valores faltantes
- Cambio nombres columnas
- Cambio magnitudes

4. DATACAMP: WRITTING EFFICIENT CODE WITH PANDAS

Han sido usadas funciones como .apply() o .map()

```
def cambio_a_cm(h):
    try:
        pies, cm = map(int, h.split('-'))
        return (pies * 30.48) + (cm * 2.54)
    except:
        return None

df['Altura'] = df['Altura'].apply(cambio_a_cm)
df['Peso'] = df['Peso'] * 0.453592

df.rename(columns={'Altura': 'Altura_cm', 'Peso': 'Peso_kg'}, inplace=True)

print(df[['Altura_cm', 'Peso_kg']].dtypes)
print(df.head())
```

```
df['Posicion_Principal'] = df['Posicion'].apply(lambda x: x.split('-')[0])
```

5. CÓDIGO EN R: EN LA LIMPIEZA Y ANALISIS DE DATOS

Superioridad en la comunicación visual de resultados, permitiendo una representación más clara mediante un sistema de capas.

INICIO DE CELDA: %%R

```
%%R
# Como hemos añadido la columna 'Rol' a nuestro DataFrame, ahora
# ¿Cuál es el promedio de puntos, rebotes y asistencias por cada

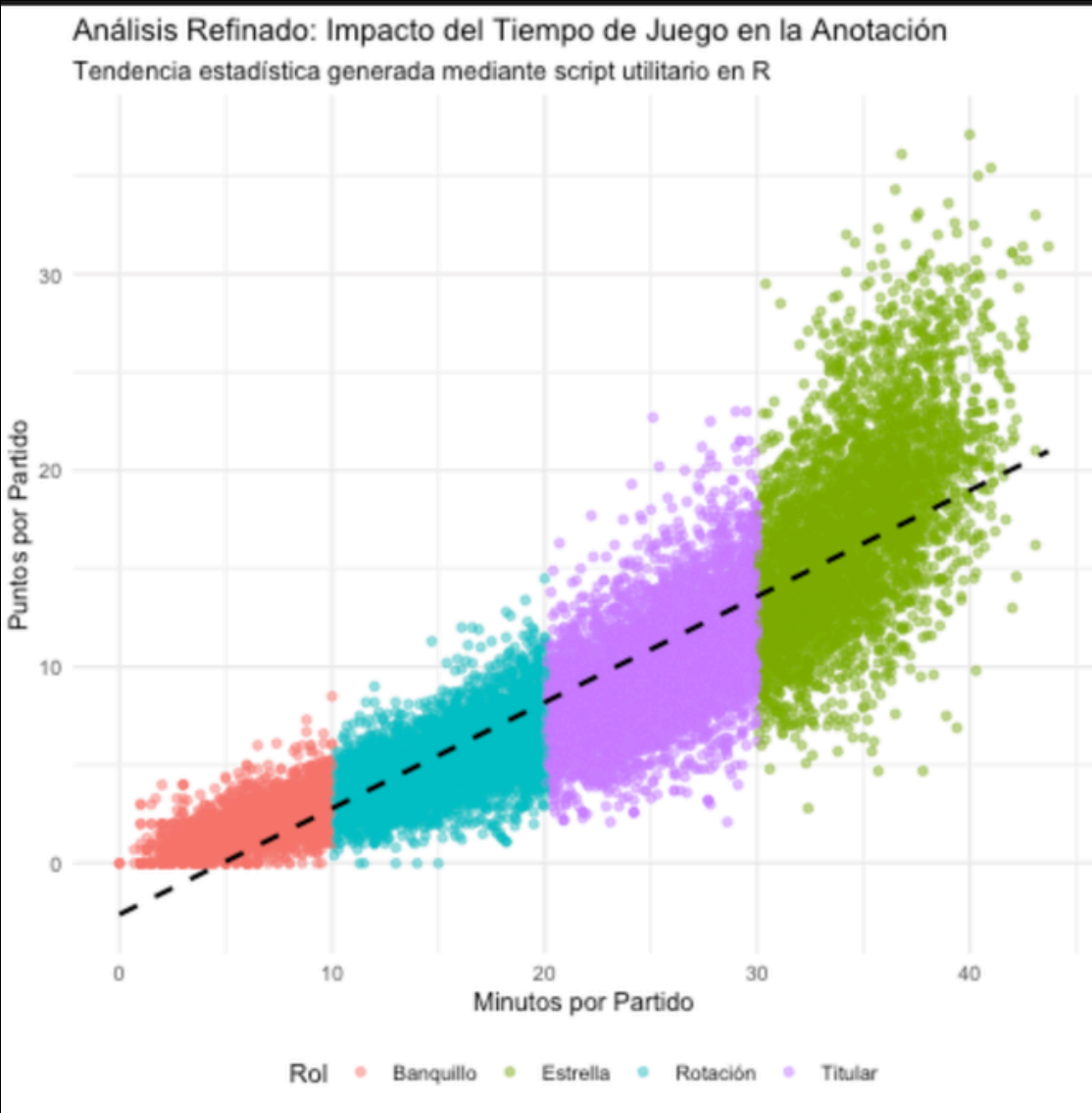
library(tidyverse)

resumen_nba <- df %>%
  group_by(Rol) %>%
  summarise(
    Total_Jugadores = n(),
    Promedio_Puntos = mean(Puntos_PP, na.rm = TRUE),
    Promedio_Asistencias = mean(Asistencias_PP, na.rm = TRUE),
    Promedio_Rebotes = mean(Rebotes_Totales_PP, na.rm = TRUE)
  ) %>%
  arrange(desc(Promedio_Puntos))

print(resumen_nba)
```

```
%%R
library(ggplot2)

ggplot(df, aes(x = Minutos_Jugados_PP, y = Puntos_PP, color = Rol)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE, linetype = "dashed", color = "black") +
  theme_minimal() +
  labs(title = "Análisis Refinado: Impacto del Tiempo de Juego en la Anotación",
    subtitle = "Tendencia estadística generada mediante script utilitario en R",
    x = "Minutos por Partido",
    y = "Puntos por Partido") +
  theme(legend.position = "bottom")
```



6. ANÁLISIS DESCRIPTIVO: VARIABLES CUANTITATIVAS

media, mediana, moda, rango, varianza, desviación típica, asimetría y el índice de Kurtosis.

```
 analisis_cuant = pd.DataFrame(index=['Media', 'Mediana', 'Moda', 'Rango', 'Varianza',  
                                     'Desviación Estándar', 'Asimetría', 'Curtosis'])
```

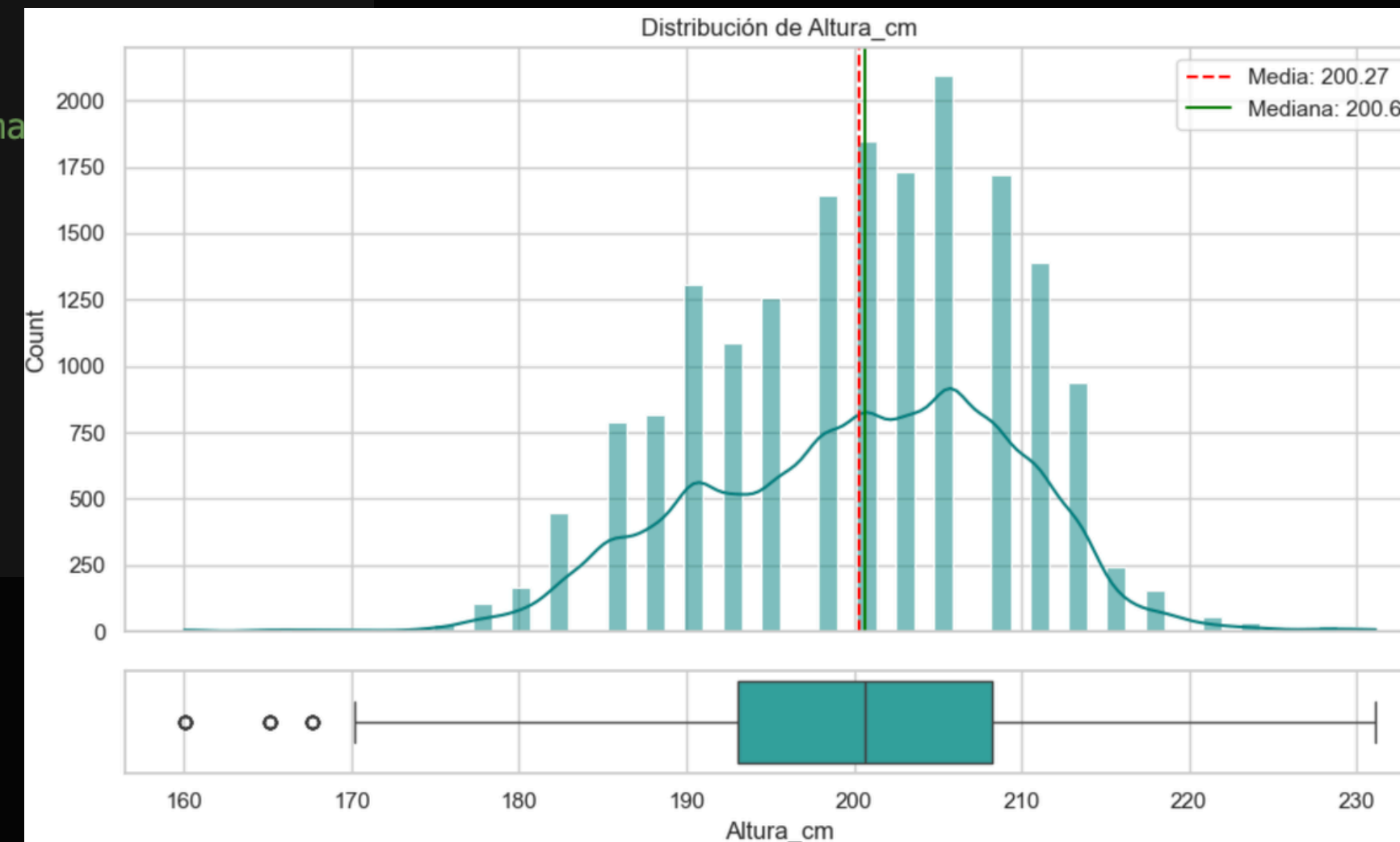
```
for col in vars_cuant:  
    analisis_cuant[col] = [  
        df[col].mean(),  
        df[col].median(),  
        df[col].mode()[0], # La moda puede devolver varios valores, toma  
        df[col].max() - df[col].min(),  
        df[col].var(),  
        df[col].std(),  
        df[col].skew(),  
        df[col].kurtosis()  
    ]
```

```
display(analisis_cuant.round(2))
```

DATA CAMP: PYTHON FOR R USERS

Uso del paquete seaborn

DISTRIBUCIONES DE CADA VARIABLE



ANÁLISIS DESCRIPTIVO: VARIABLES CUALITATIVAS

frecuencia absoluta y relativa

```
resumenes = []

for col in vars_cuali:
    if col in df.columns:

        abs_f = df[col].value_counts()
        rel_f = (df[col].value_counts(normalize=True) * 100)

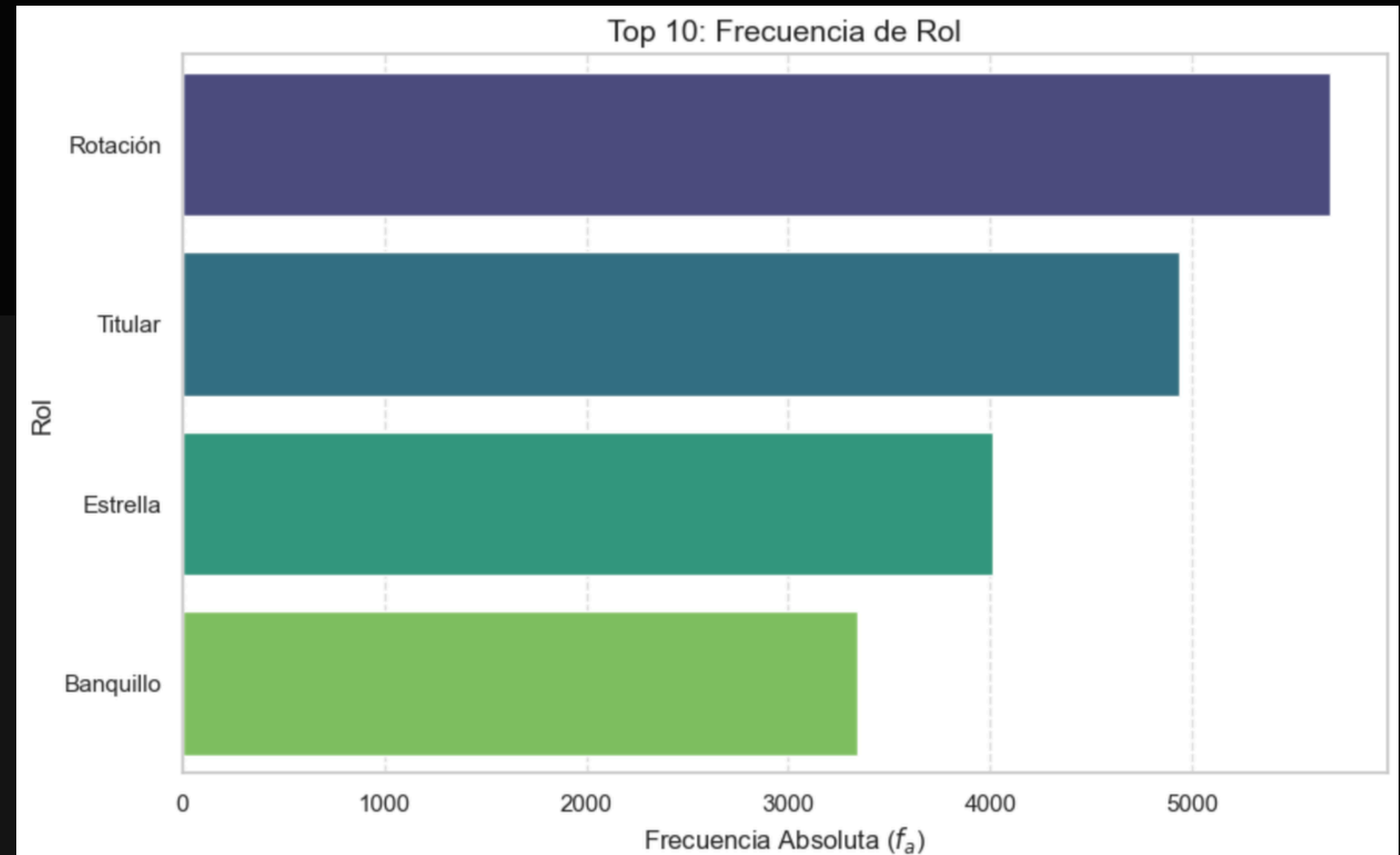
        temp_df = pd.DataFrame({
            'Frecuencia Absoluta': abs_f,
            'Frecuencia Relativa (%)': rel_f
        })

        temp_df.index.name = 'Categoría'
        temp_df['Variable'] = col

        if df[col].nunique() > 10:
            resumenes.append(temp_df.head(5).reset_index())
        else:
            resumenes.append(temp_df.reset_index())

tabla_maestra_cuali = pd.concat(resumenes).set_index(['Variable', 'Categoría'])

display(tabla_maestra_cuali.round(2))
```



7. DATACAMP: RESHAPING DATA WITH PANDAS

ordenación : conocer los líderes de cada categoría

```
print("--- TOP 10 HISTÓRICO: PORCENTAJE DE TIROS ---")
top_historico = df.sort_values(by='Porcentaje_Tiros', ascending=False).head(10)
display(top_historico[['Jugador', 'Equipo', 'Temporada', 'Porcentaje_Tiros', 'Puntos_PP']])

# 2. Líderes por Temporada (
print("\n--- LÍDERES POR TEMPORADA: VICTORIAS ---")
idx_lideres_temp = df.groupby('Temporada')['Victorias'].idxmax()
lideres_por_temporada = df.loc[idx_lideres_temp]

lideres_por_temporada = lideres_por_temporada.sort_values(by='Temporada')
display(lideres_por_temporada[['Temporada', 'Jugador', 'Equipo', 'Victorias']])

# 3. El "Top 3" por Posición
print("\n--- TOP 3 ANOTADORES POR POSICIÓN ---")
top_por_posicion = df.sort_values(['Posicion_Principal', 'Porcentaje_Tiros'], ascending=[True, False])
top_por_posicion = top_por_posicion.groupby('Posicion_Principal').head(3)
display(top_por_posicion[['Posicion_Principal', 'Jugador', 'Porcentaje_Tiros', 'Puntos_PP']])
```

8. DEEP LEARNING: ARQUITECTURA DE AUTOENCODER

5 PASOS:

- . Filtrado de Calidad
- . Ponderación Estratégica
- . Extracción de Características (Autoencoder)
- . Cálculo de Similitud
- . Proyección de Impacto

```
# 4. FUNCIÓN DE DEEP LEARNING
def obtener_recomendacion_dl(nombre, temporada):
    # Filtro por posición y año
    perfil_lesionado = df[(df['Jugador'] == nombre) & (df['Temporada'] == temporada)].iloc[0]
    posicion = perfil_lesionado['Posicion_Principal']
    df_pool = df[(df['Posicion_Principal'] == posicion) & (df['Temporada'] == temporada)].copy().reset_index(drop=True)

    umbral_puntos = perfil_lesionado['Puntos_PP'] * 0.7
    df_pool = df_pool[df_pool['Puntos_PP'] >= umbral_puntos].copy().reset_index(drop=True)

    if nombre not in df_pool['Jugador'].values:
        fila_jugador = df[(df['Jugador'] == nombre) & (df['Temporada'] == temporada)]
        df_pool = pd.concat([df_pool, fila_jugador], ignore_index=True)

    # Atributos para la Red Neuronal
    features = ['Edad', 'Partidos_Jugados', 'Minutos_Jugados_PP', 'Puntos_PP', 'Asistencias_PP', 'Rebotes_Totales_PP']

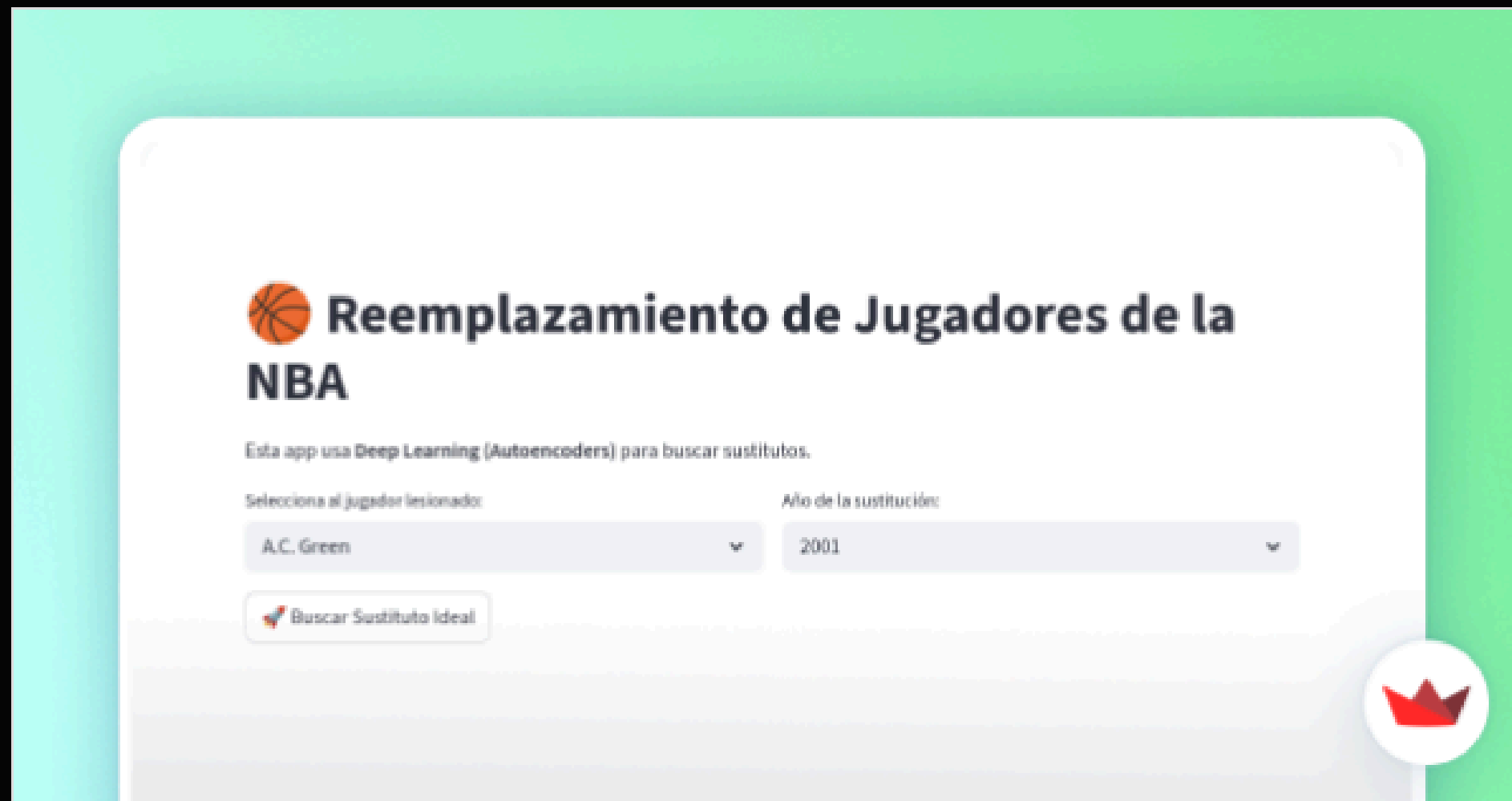
    scaler = StandardScaler()
    X = scaler.fit_transform(df_pool[features])
    pesos = np.array([0.1, 1.0, 3.5, 10.0, 5.0, 5.0])
    X = X * pesos

    input_dim = len(features)
    input_layer = Input(shape=(input_dim,))
    encoded = Dense(8, activation='relu')(input_layer)
    encoded = Dense(4, activation='relu')(encoded)
    decoded = Dense(8, activation='relu')(encoded)
    output_layer = Dense(input_dim, activation='sigmoid')(decoded)

    autoencoder = Model(input_layer, output_layer)
    encoder = Model(input_layer, encoded)
    autoencoder.compile(optimizer='adam', loss='mse')
```


9. STREAMLIT

```
(Trabajo_IAE) PS C:\Users\Pablo\Documents\testuv\Inteligencia_Artificial\Trabajo_IAE> uv run streamlit run appdl.py
```



Herramienta para Reemplazar Jugadores

OBJETIVO: El objetivo del proyecto es que, dado un jugador y un año concretos, se pueda proporcion...

Streamlit